

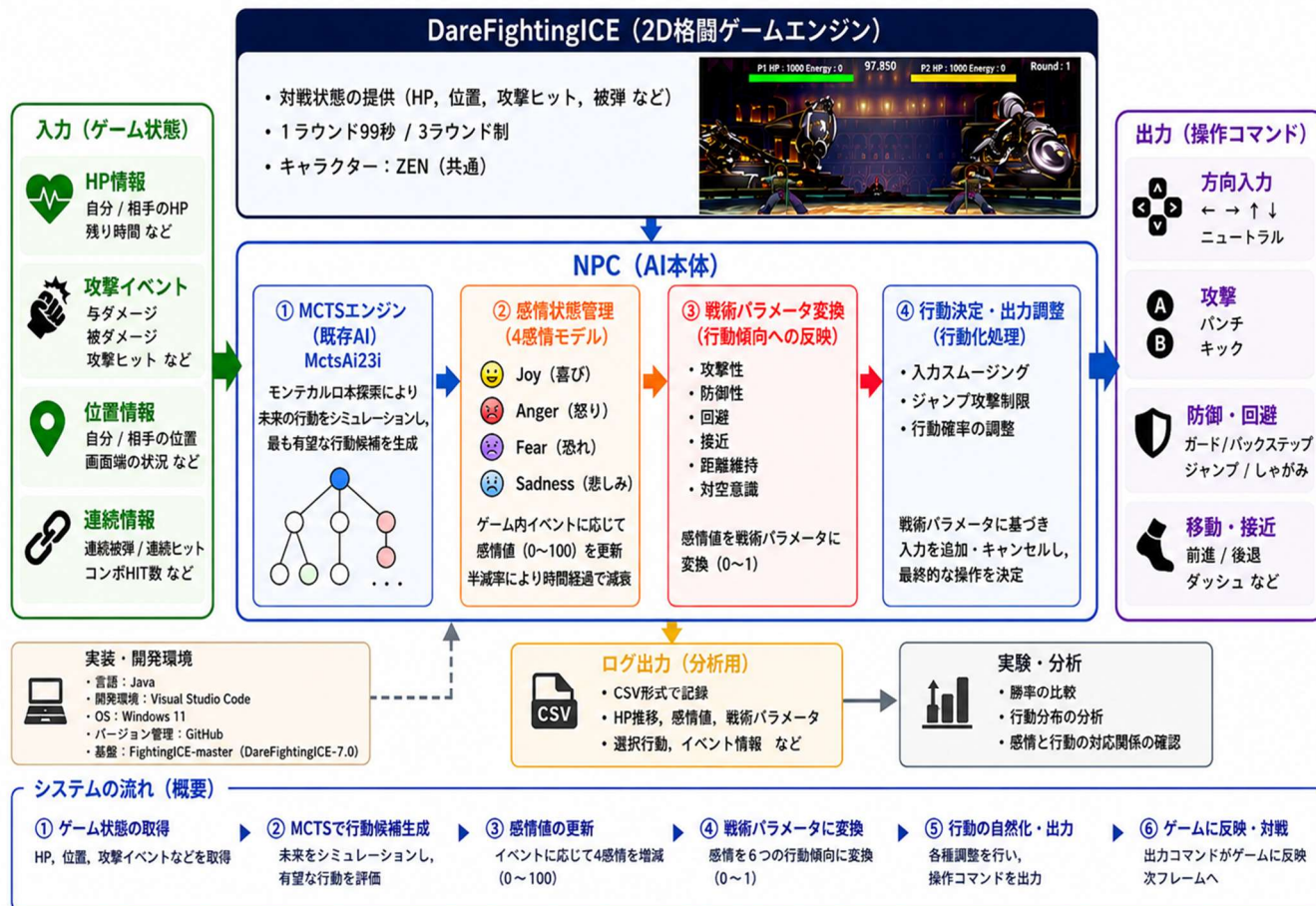
氏名：櫻井 絢翔（さくらい あやと）

出身：愛知淑徳大学 人間情報学部人間情報学科

感情モデルを導入した 2D 格闘ゲーム AI の開発

システム構成図（感情モデル搭載NPC：MctsAi23iEmotion）

ゲーム内の状況に応じて感情を変化させ、戦術パラメータへ反映することで、人間らしい行動傾向を実現



1. 研究概要

本研究では、2D 格闘ゲームプラットフォーム「DareFightingICE」を用いて、NPC (Non-Player Character) が人間のように状況へ反応し、行動を変化させる AI の開発に取り組みました。従来のゲーム AI は、勝率を最大化するために高速で最適解を探索する一方で、

- ・単調な行動パターン
- ・人間には不可能な反応速度
- ・感情の揺らぎがない不自然な挙動

といった課題がありました。

そこで本研究では、心理学の理論である プルチックの感情モデル と ラッセルの感情円環モデル を参考に、NPC が以下の 4 つの感情を内部で持つ仕組みを設計しました。

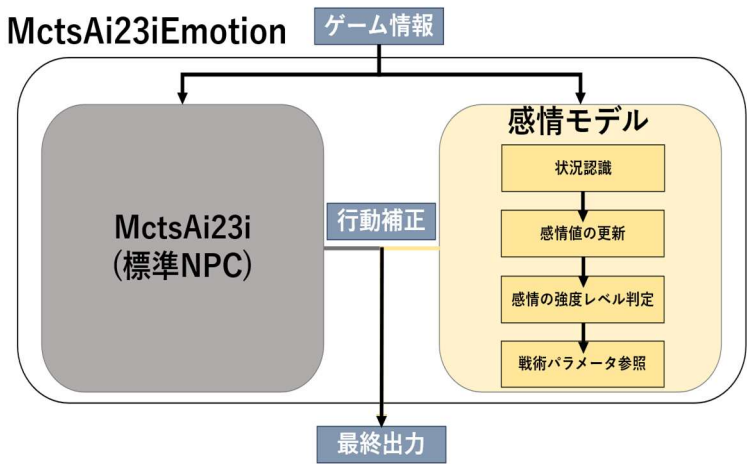
- ・喜び・怒り・恐れ・悲しみ

ゲーム中に発生するデータ（与ダメージ、被ダメージ、体力差、危険エリアでの拘束時間など）をリアルタ

イムに読み取り、 これらをトリガーとして感情値を数値化し、更新します。さらに、算出した感情値をもとに、「攻める／守る／距離を取る／近づく／回避する」といったように状況に応じて行動比率を自動調整します。これにより、NPC は固定的な行動ではなく、状態変化に応じて柔軟に戦術を切り替えることができます。このアプローチにより、 従来の「最適解だけを選ぶ AI」ではなく、 人間のように揺らぎを持ち、状況に応じて戦術を変える NPC を実現しました。

2. システム構成図

本研究で開発した AI（MctsAi23iEmotion）は、既存のベース AI（MctsAi）に「感情モデル」を組み合わせた以下の構造になっています。



具体的には、既存の MCTS アルゴリズムをそのまま活かしつつ、その内部に「感情更新ロジック」と「行動補正フィルタ」を追加する形で設計しました。 これにより、コアの探索アルゴリズムの安定性を保ちながら、NPC が状況と内部状態（感情）の両方を踏まえて行動を確率的に変化させることが可能になっています。

図 1 感情モデル導入 AI の内部構成

3. 使用技術・実行環境

開発言語	Java (java version 24.0.1)
実装基盤	DareFightingICE-7.0 / FightingICE-master (立命館大学 ICE Lab 開発)
基幹アルゴリズム	モンテカルロ木探索 (MCTS) 「今この行動をすると、少し先の未来がどうなるか」を高速でシミュレーションし、最も有利になる行動を選ぶ仕組み
使用心理学	・プルチックの感情の輪 (Joy, Anger, Fear, Sadness) ・ラッセルの感情円環モデル (快-不快: Valence / 覚醒-非覚醒: Arousal)

開発・管理環境	Visual Studio Code / GitHub
実行環境仕様	Windows 11 Home, Core i7-11700 (8C/16T) , RAM 16.0GB, GPU RTX 3070

4. 工夫した点

本研究では、NPC がより人間らしい行動をとれるようにするため、以下の3点を中心にシステム設計を工夫しました。

① 感情データを2軸にまとめ、処理を軽量化

NPC が持つ4つの感情（喜び・怒り・恐れ・悲しみ）をそのまま扱うと、条件分岐が複雑になり、処理負荷や矛盾が発生しやすくなります。そこで、心理学の理論を応用し、

- ・Valence（快・不快）＝ 喜び - 悲しみ
- ・Arousal（覚醒度）＝ 怒り - 恐れ という2軸の数値に統合しました。

これにより、感情が競合した際の矛盾の解消を実現しました。

② 感情の持続時間を調整し、不自然な急変を防止

人間の感情は一瞬で切り替わるのではなく、しばらく残りながら徐々に薄れていきます。この特徴を再現するため、感情ごとに異なる減衰時間を設定しました。（喜び：3秒、恐れ：4秒、怒り：5秒、悲しみ：6秒）

これにより、瞬間的な出来事に過剰反応しない継続的な状況変化”を正しく検知できるという、より安定した感情変化を実現しました。

③ 感情を行動へなだらかに反映させる戦術パラメータによる確率的制御

本研究では、感情がそのまま行動に直結してしまうと動きが急激に変わり不自然になるため、まず感情値を「攻撃性」や「防御性」といった戦術パラメータに変換し、0.0～1.0の連続値（0.5が中立）として管理する仕組みを導入しました。戦術パラメータの値に応じてAIが出力した操作を確率的に追加・キャンセルする制御を行い、例えば攻撃性が低い場合は攻撃入力を一定確率で取り消し、高い場合は攻撃がキャンセルされにくくなるよう調整しました。この確率的な揺らぎによって、不安で攻めが弱くなる、怒りで手数が増える、迷いで行動がぶれるといった、人間が感情によって少しずつ行動傾向を変えていく自然な変化を再現します。

④ルールベースとMCTSのハイブリッド設計による、開発コスト抑制と制御性の両立

本研究では、膨大な対戦データを集めて学習させる必要がある機械学習方式ではなく、既存の強力な思考アルゴリズムであるMCTSと、心理学モデルに基づくルールベースを組み合わせたハイブリッド設計を採用しました。試合中の最適な戦術判断はMCTSに任せつつ、その出力に対して感情ルールに基づく確率的な補正を後段で加えることで、AIの強さを保ちながら挙動の予測不能さを排除した。この仕組みにより、再学習の必要がなく開発コストを抑えられるだけでなく、設計者が意図した「人間らしい思考選択」や「感情による揺らぎ」を確実に反映可能にしました。

5. 問題解決例

・問題 1：起動エラーの原因特定と環境構築の復旧（環境構築時の課題）

【問題】 GitHub より最新プラットフォームをダウンロードして環境構築を行った際、ゲームが起動せず、画面が表示されない状態だった。

【原因】 ソースコードの読み込み処理とディレクトリ構成を調査したところ、GitHub 版は軽量化のため最小構成となっており、キャラクターの画像やモーションデータを格納する「data/」フォルダー一式が除外されている構造であることが判明。そのため、必要なリソースが読み込めず起動エラーが発生していた。

【解決策】 過去のアーカイブ版（DareFightingICE-7.0）を調査し、不足している data/ ディレクトリ一式を抽出して最新版へ移植。依存関係を正しく復元することで、起動時のエラーを完全に解消し、正常にゲーム画面が表示される環境を構築した。

問題 2：AI の拡張方式の見直しと設計改善

【問題】 既存 AI（MCTS ベース）に対して、外部から別 AI モジュールを重ねて制御する外部連携方式を検証したが、複数の処理が同時に動くことで、処理の順番が乱れたり内部データが食い違ったりし、AI の状態が壊れてしまう状況が発生していた。

【原因】 既存 AI の内部構造を詳しく調べたところ、既存 AI が外部からの動的な介入を前提としない構造であることが判明した。どこか一部分だけを変更すると、本来は順番に流れるはずの処理の道が途中で途切れてしまい、内部の整合性が崩れてシステムが止まってしまうことが分かった。これは、設計書のない既製ライブラリが強く結びついたまま作られていたため、外側から無理に手を入れると内部のバランスが壊れてしまうという構造的な問題が原因だと考えられる。

【解決策】 そこで発想を切り替え、既存 AI を外側から操作するのではなく、中身を理解したうえで正しく継承・流用し、内部で安全に拡張する方式に変更した。

具体的には、

- 既存 AI をそのまま基盤として受け継ぐ
- コアとなる探索処理には触れずに利用
- 追加したい感情制御の仕組みを内部に組み込む形でまとめ直した

という設計に作り替えた。

この方法により、既存 AI の安定性を保ちながら、新しい機能を安全に追加できる構造を実現できた。

問題 3：AI の評価値偏りに対するペナルティ機構の設計

【問題】 既存 AI が特定の「空中ジャンプ攻撃」ばかりを執拗に連発する傾向があった。提案 AI を導入しても行動の偏重は変わらず、本来の目的であった「感情の変化に応じた地上戦の駆け引き」などの多様な動きが画面上にまったく現れず、システムの動作に極端な偏りが出てしまった。

【原因】 コア AI が実装している MCTS の評価関数をログ解析したところ、シミュレータが想定する先の未来予測において、「ジャンプ攻撃」が攻撃性能・回避性能の双方において局所的な最適解になりやす

く、評価値が特定の行動に偏る「AI の最適な行動パターンへの固定化（過学習に似た状態）」が起きていることが分かった。

【解決策】 ジャンプ行動が連続した場合に評価を下げる独自のペナルティ機構（20 回制限・評価値減算）を実装し、特定行動の選択確率を意図的に減衰させる仕組みを導入した。この調整により、AI が地上戦を含む多様な行動を取るようになり、感情パラメータによる行動変化が画面上で明確に確認できるようになった。

問題 4：処理による挙動のカクツキの平滑化

【問題】 感情 AI モデルを統合した検証で、キャラクターが画面上で小刻みに震える（左右へ超高速で揺れ続ける）という不自然な挙動が発生した。リアルタイム処理が過敏に反応しすぎた結果、人間らしさとはかけ離れた不自然な動きになってしまっていた。

【原因】 内部ログを解析したところ、想定よりも感情パラメータが激しく揺れ動いており、1/60 秒ごとにわずかな変化が起きるたびに方向入力が即座に切り替わっていることが分かった。その結果、「右→左→右」といった入力が人間では不可能な速度で連続発生し、制御が細かく切り替わり続ける状態に陥っていた。この過敏な反応が積み重なり、キャラクターが震えるように見える不自然な挙動を引き起こしていた。

【解決策】 急激な入力変化を抑え、人間らしい滑らかな動きを再現するため、方向入力を一定時間保持する処理を実装した。入力切り替えを検知すると専用カウンタを起動し、数秒の間は前回の入力を維持することで、超高速で発生するノイズ的な入力を省いた。これにより、キャラクターの移動が滑らかになり、不自然な震えが解消された。

6. 実験および分析結果（システムの動作検証と評価）

本研究では、開発した感情モデルが実際に NPC の行動へ反映されているかを確認するため、既存の MctsAi23i と提案 AI である MctsAi23iEmotion を比較しながら動作検証を行いました。

実験目的

本実験では、以下の 2 点を検証することを目的としました。

- ・感情モデルが実際の行動選択へ反映されているか
- ・感情モデルの導入によって行動の多様性が向上しているか

実験方法

DareFightingICE 環境において、既存 AI と提案 AI を複数回対戦させ、勝敗結果および行動ログを収集しました。また、試合中の感情値（喜び・怒り・恐れ・悲しみ）、HP 差、選択行動などを CSV 形式で記録し、感情状態と行動傾向の関係を分析しました。

実験結果

対戦カード	P1（左側）	P2（右側）	P1 勝率	P2 勝率
MctsAi23 vs MctsAi23	MctsAi23	MctsAi23	40.00%	60.00%
MctsAi23Emotion vs MctsAi23Emotion	MctsAi23Emotion	MctsAi23Emotion	53.30%	43.30%
MctsAi23Emotion vs MctsAi23	MctsAi23Emotion	MctsAi23	83.30%	16.70%
MctsAi23 vs MctsAi23Emotion	MctsAi23	MctsAi23Emotion	40.00%	60.00%
MctsAi23Emotion vs MctsAi23Emotion	MctsAi23Emotion	MctsAi23Emotion	53.30%	43.30%

図 2 標準 AI と提案 AI の対戦結果比較

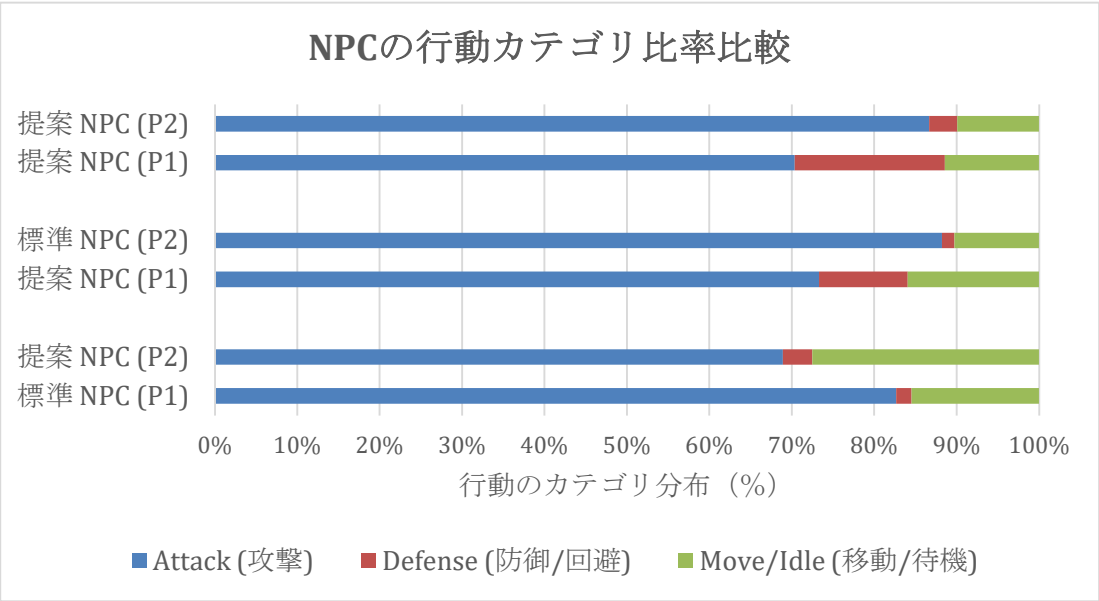


図 3 NPC の行動カテゴリ分布比較

既存 AI と比較した結果、提案 AI は同等以上の勝率を維持しながら、状況に応じた行動変化を実現できていることを確認しました。また、ログ分析および試合映像の確認から、感情値の変化に応じて行動傾向が変化することを確認しました。

図 3 に示す行動カテゴリ比率の比較では、提案 AI は既存 AI よりも攻撃・防御・移動の割合が状況に応じて変化しており、固定的な行動パターンに依存しないことが確認できました。

特に、劣勢時には防御・回避行動が増加し、優勢時には積極的な攻撃行動が増加するなど、人間プレイヤーに近い戦術変化が見られました。

考察

従来の MCTS ベース AI は常に勝率の高い行動を優先するため、同じ状況では類似した行動を選択する傾向があります。一方で、本研究で提案した感情モデルは、MCTS の探索結果に対して感情状態に応じた補正を加えることで、同一状況であっても異なる行動を選択する可能性を持たせています。その結果、AI としての競争力を維持しながら、より人間らしい戦術変化や行動の多様性を実現できました。

本研究を通じて得られた知見

本研究では、既存システムの解析から設計変更、機能追加、ログ分析、性能評価までを一貫して実施しました。特に、既存 AI を安全に拡張するための設計改善や、問題発生時の原因分析と解決策の検討を通じて、ソフトウェア開発における設計力・調査力・問題解決力を身につけることができました。

7. 成果物の添付・GitHub リポジトリの構成案

本研究のソースコードは GitHub 上にて管理し、添付する成果物（リポジトリ）内で主に変更・追加を行ったのは以下の通りです。

ディレクトリ / ファイル	役割
src/ai/MctsAi23iEmotion.java	主ソースコード（提案 AI）：感情計算、ラッセル 2 軸変換、戦術パラメータのリアルタイム動的補正ロジックを 1 ファイルに集約。
src/ai/CsvLogger.java	ログ収集・監視モジュール：対戦中の時間、体力、感情値、選択行動、勝敗を CSV ファイルに格納・出力する。
data/AIList.csv	システム構成定義ファイル：作成した jar ファイルをランタイム環境に認識させるためのプラットフォーム設定ファイル。
run.bat	起動用スクリプト：ゲーム内のタイマーやキャラクターの初期体力などの設定を柔軟に変更してゲームを起動できるスクリプト。

【GitHub 公開時の URL・アクセス方法について】

本研究で開発したソースコードおよび実行環境一式は、以下の URL か itHub リポジトリにて一般公開しております。

URL : <https://github.com/gssakurai0824-create/mcts-emotionNPC>